

Después de cubrir **Observer** (notificación push), **Strategy** (algoritmos intercambiables) y **Command** (acciones encapsuladas), el siguiente patrón de comportamiento más valioso para que ustedes ingenieros exploren de manera **autodidacta** es el patrón **State**.

Patrón State (Estado)

Instrucciones para el Estudiante

"Esta guía está diseñada para que trabajes de manera independiente durante la Semana 15. Sigue cada sección, implementa los ejemplos y responde las preguntas de reflexión. Al final, encontrarás una actividad para integrar State en el proyecto integrador del Sistema de Gestión Académica."

1. Introducción y Definición

¿Qué es el patrón State?

El patrón **State** permite a un objeto **alterar su comportamiento** cuando su estado interno cambia. El objeto parecerá cambiar de clase.

Categoría: Patrón de Comportamiento (Behavioral).

Objetivo Principal: Encapsular los comportamientos específicos de cada estado en clases separadas, evitando condicionales complejos (if-else o switch) que verifican el estado actual.

Cita Clave (GoF): *"Permite a un objeto alterar su comportamiento cuando su estado interno cambia. El objeto parecerá cambiar de clase."* — Gamma, Helm, Johnson, Vlissides (1994).

2. El Problema que Resuelve

Sin el patrón State (Código espagueti)

Imagina un **sistema de solicitud de préstamo** en la biblioteca académica. Una solicitud puede estar en diferentes estados:

- **Pendiente** (recién creada)
- **Aprobada** (el bibliotecario la aceptó)

- **Rechazada** (no cumple requisitos)
- **Prestado** (el libro fue entregado)
- **Devuelto** (el libro fue regresado)

Sin State, el código sería algo así:

java

```
class SolicitudPrestamo {

    private String estado; // "pendiente", "aprobada", "rechazada", "prestado", "devuelto"

    void aprobar() {

        if (estado.equals("pendiente")) {

            System.out.println("Solicitud aprobada");

            estado = "aprobada";

        } else if (estado.equals("aprobada")) {

            System.out.println("Ya estaba aprobada");

        } else if (estado.equals("rechazada")) {

            System.out.println("No se puede aprobar una solicitud rechazada");

        } else if (estado.equals("prestado")) {

            System.out.println("El préstamo ya está activo");

        } else if (estado.equals("devuelto")) {

            System.out.println("El proceso ya terminó");

        }

    }

    void rechazar() { /* otro if-else gigante */ }

    void prestarLibro() { /* otro if-else gigante */ }

    void devolverLibro() { /* otro if-else gigante */ }

}
```

Problemas:

- Cada método tiene una estructura if-else o switch enorme.
- Al añadir un nuevo estado (ej. "Vencido"), hay que modificar TODOS los métodos.
- Violación del principio **Open/Closed**.

- Código difícil de leer, probar y mantener.

Con el patrón State

Cada estado es una **clase separada** que sabe qué acciones puede realizar y a qué nuevo estado transicionar.

3. Analogías para una Comprensión Intuitiva

Analogía 1: El Botón de Reproducción de un Reproductor de Música

- **Sujeto:** El reproductor de música (contexto).
- **Estados:** Detenido, Reproduciendo, Pausado.
- **Comportamiento:** El botón "Play" se comporta diferente según el estado:
 - Si está **Detenido** → Comienza a reproducir.
 - Si está **Reproduciendo** → No hace nada o pausa.
 - Si está **Pausado** → Reanuda la reproducción.

El botón no necesita if-else. El propio estado sabe qué hacer.

Analogía 2: El Semáforo de una Intersección

- **Contexto:** El semáforo.
- **Estados:** Rojo, Amarillo, Verde.
- **Comportamiento:** La acción "cambiarSiguiente()" se comporta diferente:
 - **Rojo** → Cambia a Verde.
 - **Verde** → Cambia a Amarillo.
 - **Amarillo** → Cambia a Rojo.

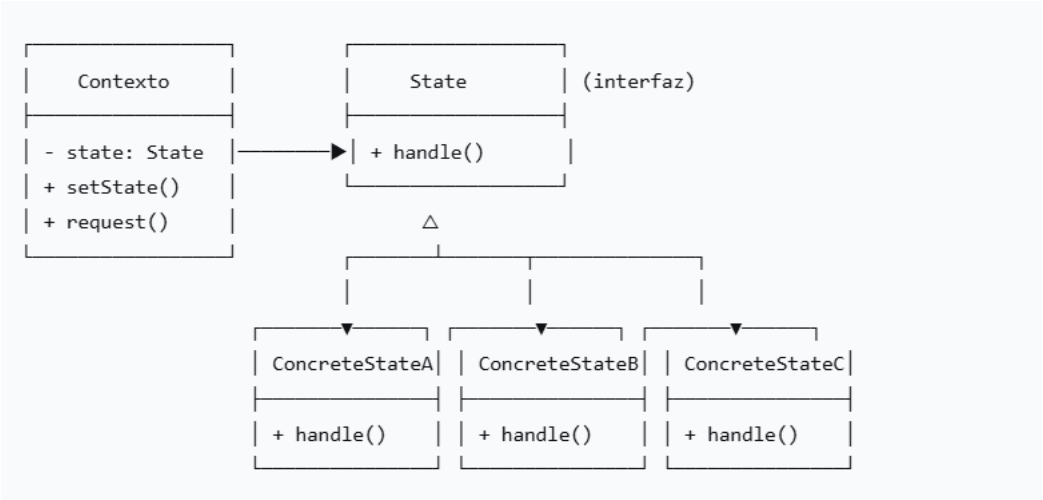
Cada estado conoce su transición.

Analogía 3: Un Pedido en Rappi / Mercado Libre

- **Contexto:** El pedido.
- **Estados:** Recibido, Confirmado, En preparación, En camino, Entregado, Cancelado.

- **Comportamiento:** La acción "cancelar()" es válida solo en algunos estados (Recibido, Confirmado). En "Entregado" o "Cancelado", no tiene sentido.

4. Estructura Técnica (Diagrama Lógico)



Componentes del patrón:

Componente	Rol
Contexto	Objeto que tiene un estado interno. Delega el comportamiento a su objeto State actual.
State (Interfaz)	Declara métodos comunes para todos los estados concretos.
ConcreteState	Implementa comportamientos específicos para un estado y define transiciones a otros estados.

5. Ejemplo de Código (Java)

Caso: Solicitud de Préstamo en Biblioteca Académica

```
java
// 1. Interfaz State
interface EstadoSolicitud {
```

```

    void aprobar(SolicitudPrestamo contexto);
    void rechazar(SolicitudPrestamo contexto);
    void prestarLibro(SolicitudPrestamo contexto);
    void devolverLibro(SolicitudPrestamo contexto);
}

// 2. Contexto (la solicitud que cambia de estado)
class SolicitudPrestamo {
    private EstadoSolicitud estadoActual;
    private String estudiante;
    private String libro;
    public SolicitudPrestamo(String estudiante, String libro) {
        this.estudiante = estudiante;
        this.libro = libro;
        this.estadoActual = new EstadoPendiente(); // Estado inicial
    }
    void setEstado(EstadoSolicitud nuevoEstado) {
        this.estadoActual = nuevoEstado;

        System.out.println(" 🚩 Estado cambiado a: " + nuevoEstado.getClass().getSimpleName());
    }

    // Métodos que delegan al estado actual
    void aprobar() { estadoActual.aprobar(this); }
    void rechazar() { estadoActual.rechazar(this); }
    void prestarLibro() { estadoActual.prestarLibro(this); }
    void devolverLibro() { estadoActual.devolverLibro(this); }

    // Getters
    public String getEstudiante() { return estudiante; }
    public String getLibro() { return libro; }
}

// 3. Estados Concretos
class EstadoPendiente implements EstadoSolicitud {
    public void aprobar(SolicitudPrestamo contexto) {
        System.out.println(" 🟢 Solicitud APROBADA para " + contexto.getEstudiante());
        contexto.setEstado(new EstadoAprobada());
    }
}

```

```

    }

    public void rechazar(SolicitudPrestamo contexto) {

        System.out.println("❌ Solicitud RECHAZADA para " + contexto.getEstudiante());
        contexto.setEstado(new EstadoRechazada());
    }

    public void prestarLibro(SolicitudPrestamo contexto) {

        System.out.println("⚠️ No se puede prestar: la solicitud está PENDIENTE de aprobación");
    }

    public void devolverLibro(SolicitudPrestamo contexto) {

        System.out.println("⚠️ No se puede devolver: la solicitud está PENDIENTE");
    }
}

class EstadoAprobada implements EstadoSolicitud {

    public void aprobar(SolicitudPrestamo contexto) {

        System.out.println("⚠️ La solicitud ya estaba aprobada");
    }

    public void rechazar(SolicitudPrestamo contexto) {

        System.out.println("⚠️ No se puede rechazar una solicitud ya aprobada");
    }

    public void prestarLibro(SolicitudPrestamo contexto) {

        System.out.println("📖 Libro " + contexto.getLibro() + " PRESTADO a " + contexto.getEstudiante());
        contexto.setEstado(new EstadoPrestado());
    }

    public void devolverLibro(SolicitudPrestamo contexto) {

        System.out.println("⚠️ No se puede devolver: primero debe prestarse el libro");
    }
}

class EstadoPrestado implements EstadoSolicitud {

    public void aprobar(SolicitudPrestamo contexto) {

        System.out.println("⚠️ El préstamo ya está activo, no se puede aprobar de nuevo");
    }

    public void rechazar(SolicitudPrestamo contexto) {

```

```

        System.out.println(" ⚠️ No se puede rechazar un préstamo activo");
    }

    public void prestarLibro(SolicitudPrestamo contexto) {
        System.out.println(" ⚠️ El libro ya está prestado");
    }

    public void devolverLibro(SolicitudPrestamo contexto) {
        System.out.println("Libro DEVUELTO por " + contexto.getEstudiante());
        contexto.setEstado(new EstadoDevuelto());
    }
}

class EstadoDevuelto implements EstadoSolicitud {
    public void aprobar(SolicitudPrestamo contexto) {
        System.out.println("El proceso ya terminó (libro devuelto)");
    }

    public void rechazar(SolicitudPrestamo contexto) {
        System.out.println("El proceso ya terminó");
    }

    public void prestarLibro(SolicitudPrestamo contexto) {
        System.out.println("No se puede prestar: el libro ya fue devuelto");
    }

    public void devolverLibro(SolicitudPrestamo contexto) {
        System.out.println("El libro ya estaba devuelto");
    }
}

class EstadoRechazada implements EstadoSolicitud {
    public void aprobar(SolicitudPrestamo contexto) {
        System.out.println("No se puede aprobar una solicitud rechazada");
    }

    public void rechazar(SolicitudPrestamo contexto) {
        System.out.println("La solicitud ya estaba rechazada");
    }

    public void prestarLibro(SolicitudPrestamo contexto) {
        System.out.println("No se puede prestar: solicitud rechazada");
    }
}

```

```

    }

    public void devolverLibro(SolicitudPrestamo contexto) {
        System.out.println("No se puede devolver: solicitud rechazada");
    }
}

// 4. Cliente (Demostración)
public class BibliotecaDemo {
    public static void main(String[] args) {
        SolicitudPrestamo solicitud = new SolicitudPrestamo("Carlos Pérez", "Patrones de Diseño");
        System.out.println("=== Flujo normal ===\n");
        solicitud.prestarLibro(); // No puede (está pendiente)
        solicitud.aprobar();      // Pendiente → Aprobada
        solicitud.prestarLibro(); // Aprobada → Prestado
        solicitud.devolverLibro(); // Prestado → Devuelto
        solicitud.devolverLibro(); // Ya estaba devuelto
        System.out.println("\n=== Flujo con rechazo ===\n");
        SolicitudPrestamo solicitud2 = new SolicitudPrestamo("Ana Gómez", "Clean Code");
        solicitud2.aprobar();      // Pendiente → Aprobada
        solicitud2.rechazar();     // No puede (ya aprobada)
        SolicitudPrestamo solicitud3 = new SolicitudPrestamo("Luis Mora", "Refactoring");
        solicitud3.rechazar();     // Pendiente → Rechazada
        solicitud3.prestarLibro(); // No puede (rechazada)
    }
}

```

Salida esperada:

=== Flujo normal ===

No se puede prestar: la solicitud está PENDIENTE de aprobación

Solicitud APROBADA para Carlos Pérez

Estado cambiado a: EstadoAprobada

Libro 'Patrones de Diseño' PRESTADO a Carlos Pérez

Estado cambiado a: EstadoPrestado

Libro DEVUELTO por Carlos Pérez

Estado cambiado a: EstadoDevuelto

El libro ya estaba devuelto

6. Ventajas y Desventajas

Ventajas

Ventaja	Explicación
Principio Open/Closed	Puedes añadir nuevos estados sin modificar los existentes ni el contexto.
Elimina condicionales	Adiós a enormes if-else o switch que verifican el estado.
Código más limpio	Cada estado tiene su propia clase, fácil de entender y probar.
Transiciones explícitas	Las reglas de cambio de estado están claras dentro de cada clase.

Desventajas

Desventaja	Explicación
Explosión de clases	Si hay muchos estados, tendrás muchas clases pequeñas.
Complejidad inicial	Para casos simples, un switch puede ser más rápido de implementar.

Desventaja	Explicación
Estados acoplados	Un estado conoce a otros (para transiciones), lo que genera dependencias.

7. Comparativa con Otros Patrones de Comportamiento

Patrón	Diferencia Clave con State
Strategy	Strategy cambia el algoritmo completo del contexto. State cambia el comportamiento según el estado interno . En State, el propio estado puede decidir el próximo estado.
Observer	Observer notifica a múltiples observadores cuando algo cambia. State cambia el comportamiento interno del propio objeto.
Command	Command encapsula una acción (una sola vez). State encapsula el comportamiento de un objeto que evoluciona a lo largo del tiempo.

Regla nemotécnica:

- **Strategy** → "El contexto elige qué hacer".
- **State** → "El contexto es quién es, por eso actúa diferente".

8. Aplicación en el Proyecto Integrador (Sistema de Gestión Académica)

Caso 1: Flujo de Aprobación de un Proyecto de Grado

java

// Estados: PropuestaEnviada, Revisión, Aprobada, CorrecciónRequerida, Finalizado

Caso 2: Estado de una Matrícula

java

// Estados: Preliminar, Confirmada, Pagada, Cancelada, Vencida

Caso 3: Entrega de Tareas por parte del Estudiante

java

// Estados: SinEntregar, Entregada, Calificada, Retroalimentada, Reentregada

Ejemplo rápido para tu proyecto:

java

```
class EstadoMatricula {  
    void pagar(Matricula contexto) { /* por defecto, acción inválida */ }  
    void cancelar(Matricula contexto) { /* ... */ }  
    void confirmar(Matricula contexto) { /* ... */ }  
}  
  
class EstadoPreliminar extends EstadoMatricula {  
    void pagar(Matricula ctx) {  
        System.out.println("Pago registrado, matrícula confirmada");  
        ctx.setEstado(new EstadoConfirmada());  
    }  
    void cancelar(Matricula ctx) {  
        System.out.println("Matrícula cancelada");  
        ctx.setEstado(new EstadoCancelada());  
    }  
}
```

9. Actividad Autodidacta

Objetivo

Implementar el patrón **State** para gestionar el flujo de **solicitudes de cambio** en el Sistema propio.



Instrucciones

1. **Identifica los estados** de una solicitud:
2. **Define las acciones** que pueden ocurrir:
3. **Implementar** la solución usando State.

4. Responder:

- ¿Qué ventaja tiene State frente a usar un enum con switch para este caso?
- ¿Cómo afectaría añadir un nuevo estado "Vencida" (si pasan 15 días sin revisión)?

5. Incluir:

- Código fuente completo.
- Diagrama de clases UML (simple).
- Captura de ejecución mostrando un flujo completo ej: (Pendiente → EnRevisión → Aprobada → Ejecutada).

10. Referencias para Profundizar

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns*. Capítulo "State" (pp. 305–313).
- Refactoring.Guru. *State Pattern*. <https://refactoring.guru/design-patterns/state>
- Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). Capítulo "The State Pattern".